

TP commande d'un système sur la base d'un modèle appris

Lucas Gruss, Alexandre Hache, Mickaël Zodros

February 21, 2026

1 Notions abordées, objectifs, prérequis logiciels

1.1 Notions

- systèmes non-linéaires
- systèmes MIMO
- commande à temps discret
- apprentissage par réseau de neurones

1.2 Objectifs

- apprendre la dynamique d'un système avec un réseau de neurones.
- faire la synthèse d'un contrôleur sur la base du modèle appris.

1.3 Prérequis logiciels

- Matlab version 2020b
- installation de Casadi: se rendre sur le site <https://web.casadi.org/get/> et télécharger l'archive pour la version R2016a ou plus de Matlab. Dézipper l'archive. (l'utilisation de Casadi demande simplement de l'ajouter au `path` de Matlab avec la commande `addpath("/chemin/vers/Casadi")` puis la commande `import casadi.*`).

1.4 Rendu

- Un compte-rendu est attendu pour chaque binôme/
- Compte-rendu attendu pour le vendredi 29 Mars 23h59
- A envoyer à alexandre.hache@imt-atlantique.fr

2 Présentation du système et modélisation

Le système considéré ici est souvent utilisé comme cas d'étude et utilisé afin de tester de possibles stratégies de commande (utilisation en tant que benchmark). Il est composé de 4 réservoirs d'eau, de deux pompes et deux vannes trois voies. Chaque réservoir contient un orifice par lequel le liquide s'écoule, du réservoir 3 vers le 1, du réservoir 4 vers le 2, puis des réservoirs 1 et 2 vers une cuve de rétention (voir figure 1). Le but poursuivi est la recherche des lois de commandes capables d'asservir la hauteur d'eau dans les réservoirs 1 & 2 au travers des commandes des pompes 1 & 2.

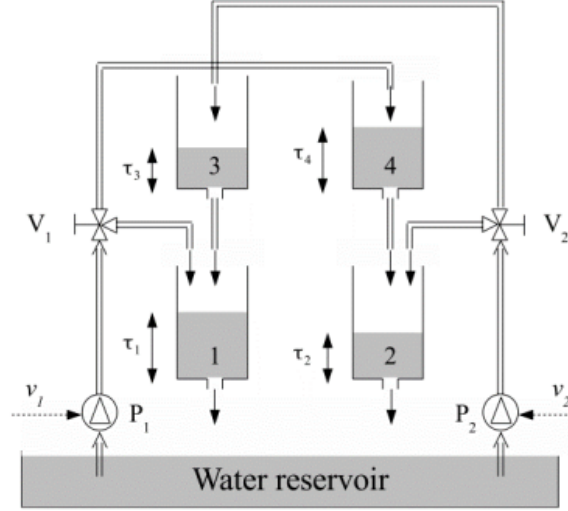


Figure 1: Schéma représentatif du système à 4 réservoirs

On pourrait modéliser ce système par une représentation d'état (potentiellement non-linéaire), telle que pour un état x du système:

$$\dot{x} = f(x, u)$$

On pourrait mener cette modélisation à partir des équations de la physique (équations de Bernoulli). Pour la simulation et la commande, on est amené à réaliser l'intégration numérique du modèle, c'est à dire approximer la solution $x(t)$ de l'équation différentielle par une suite $(x_n)_{n \in \mathbb{N}}$ telle que $x_n = x(t_n)$ où $(t_n)_{n \in \mathbb{N}}$ est une suite d'instants où l'on approxime la solution. Une méthode d'intégration numérique à un pas telle que Euler ou Runge-Kutta permet de construire la suite $(x_n)_{n \in \mathbb{N}}$ et ainsi d'obtenir un modèle d'état discret :

$$x_{k+1} = \phi(x_k, u_k)$$

On dispose effectivement de données de fonctionnement du modèle (hauteur dans les réservoirs et débits des pompes) et on propose de d'apprendre un modèle d'état discret du système en approximant la fonction ϕ par un réseau de neurones :

$$x_{k+1} = R_\theta(x_k, u_k), \quad \forall k \in \mathbb{N}, x_k \in \mathbb{R}^4, u_k \in \mathbb{R}^2$$

La coordonnée $i = 1..4$ du vecteur d'état x_k correspond à la hauteur du réservoir i et la coordonnée $j = 1..2$ du signal de commande u_k correspond au débit de commande de la pompe j . Les paramètres du réseau de neurones sont agrégés dans le vecteur $\theta \in \mathbb{R}^m$ où m est le nombre de paramètres du réseau.

3 Exploration des données d'apprentissage

Les données qui vous sont fournies correspondent à des données issues du système contrôlé par une loi de commande déjà établie pour ce système. Elles correspondent à un contrôleur discret fonctionnant à une cadence $T_s = 1s$.

1. Chargez les données dans le workspace Matlab en tapant la commande `load("data.mat")`. Le fichier `data.mat` contient toutes les données dont vous aurez besoin pour réaliser le TP.

L'apprentissage est dit *supervisé* puisque l'on dispose à la fois des entrées et des sorties du système. Les états successifs sont stockés dans la variable `states` qui est une matrice de $\mathbb{R}^{4 \times N}$, tels que la colonne i est l'état x_i . Les commandes sont stockées dans la variable `inputs` qui est une matrice de $\mathbb{R}^{2 \times N}$, telles que la colonne i est la commande u_k à l'instant $t_k = k * T_s$. Donc on a :

1. On cherche à apprendre le comportement du système, c'est-à-dire l'état x_{k+1} dans lequel il se trouve lorsqu'on lui applique la commande u_k depuis l'état x_k .
 - De quel type d'apprentissage s'agit-il ? (classification vs. regression)
 - Quelle est la fonction coût (critère d'apprentissage) qui sera minimisée au cours de l'entraînement ?
2. À partir des données qui vous sont fournies, **formez le jeu de données d'entraînement** (X, Y) . La colonne Y_i est la prédiction que doit apprendre le réseau pour une entrée X_i de sorte que $R_\theta(X_i) \approx Y_i$. Les ordres de grandeur des entrées (`inputs`) et des états mesurés (`states`) sont très différents : 10^{-4} et ≈ 1 respectivement. Pour faciliter l'entraînement, il est préférable de les ramener à la même échelle. Pour cela vous sont fournis `umax` et `xmax` (dans `data.mat`) par lesquels vous diviserez les données fournies.
3. Visualisez les données qui vous sont fournies et décrivez leurs caractéristiques (caractéristiques = choses qui vous semblent pertinentes pour l'apprentissage).
4. Séparer les données en jeux d'entraînement et de test (80/20% est une proportion courante) et mélangez aléatoirement les données. Vous pourrez utiliser la fonction `randperm`.

4 Apprentissage de la dynamique du système

4.1 Perceptron multi-couches

Le premier modèle que l'on propose de mettre en oeuvre est un perceptron multi-couches. Il est composé de 4 couches au total:

- une première couche d'entrées (6 neurones: les 4 premiers pour l'état et les 2 derniers pour le signal de commande)
 - une couche "entièrement connectée" avec n_{hidden} neurones cachés.
 - une couche correspondant à la fonction d'activation.
 - une couche de sortie à 4 neurones.
1. Exprimer, en fonction de n_{hidden} le nombre de paramètres à entraîner du réseau. On n'oubliera pas de dénombrer les biais de la première couche "entièrement connectée".

2. Implémenter, sous MATLAB, un modèle MLP à partir de l'outil Deep Network Designer. Exporter ensuite le code généré par l'application, puis personnaliser le réseau de neurones afin de tester différentes fonctions d'activation : sigmoïde, tanh et ReLU.¹ .

(a) Définissez une fonction de perte

Choisir une fonction de coût adaptée au problème traité . La perte doit comparer la sortie prédite par le réseau $\hat{y}$ à la vérité terrain y :

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

où θ représente les paramètres du réseau.

(b) Mettez en place la boucle d'entraînement

La boucle d'entraînement doit suivre les étapes suivantes :

- Initialiser les paramètres du réseau (poids et biais).
- Pour chaque époque :
 - Parcourir les données (échantillon par échantillon ou par mini-batch).
 - Effectuer une propagation avant (forward pass) pour calculer les prédictions.
 - Calculer la perte à l'aide de la fonction définie précédemment.
 - Calculer les gradients par rétropropagation (backpropagation).
 - Mettre à jour les paramètres avec un optimiseur (descente de gradient) :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

où η est le taux d'apprentissage.

- Afficher ou stocker la valeur de la perte pour suivre la convergence.

3. Entraîner ce réseau et comparer le résultat de l'entraînement pour plusieurs fonctions d'activation et nombre de neurones. ²
4. Reprenez la procédure d'entraînement en implémentant la mise à jour des paramètres à l'aide de la fonction adamupdate, plutôt qu'en utilisant l'optimiseur automatique.
5. Comparez les résultats obtenus avec ceux issus de la descente de gradient classique
6. Pour le réseau le plus performant, jugez de ses qualités pour la simulation en comparant les prédictions du réseau aux données. Cette étape diffère de la question précédente, puisque le réseau a été entraîné pour reproduire la dynamique du système à 1 pas. Ici, il s'agit de "reboucler le réseau sur-lui-même". Commentez les résultats obtenus.

¹<https://fr.mathworks.com/help/deeplearning/ug/list-of-deep-learning-layers.html>

²<https://fr.mathworks.com/help/deeplearning/ref/trainnet.html>

4.2 Approche structurée

On s'est donné pour objectif d'apprendre la dynamique discrétisée du système. On pourrait tout à fait décider d'apprendre la dynamique à temps continu à la place, c'est à dire apprendre le modèle :

$$\dot{x}(t) = f(x(t), u(t))$$

Seulement, le souci est que nous ne disposons que de données à temps discret et non pas à temps continu du système. Par ailleurs, les commandes sont bloquées pendant 1 seconde, puisque le contrôleur qui a généré les données est un contrôleur à temps discret.

Tout d'abord on fait l'hypothèse que le système est en fait linéaire vis-à-vis des entrées. On peut donc réécrire la dynamique sous la forme :

$$\dot{x} = g(x) + Bu$$

Supposons que l'on intègre numériquement ce système avec un schéma d'Euler, on a alors :

$$x_{k+1} = x_k + h\dot{x}(t_k) = x_k + hg(x_k) + hBu_k$$

On va procéder à l'apprentissage *structuré* de la dynamique du système, sous hypothèse de linéarité vis-à-vis de la commande et en cherchant la dynamique continue du système, c'est à dire un réseau $R_\theta(x_k, u_k)$ qui implémente la structure ci-dessus.³

1. Implémenter le réseau.
2. Identifiez la partie du réseau qui apprend la dynamique continue.
3. Comment appelle-t-on ce type de réseaux.
4. Comparer les performances du réseau structuré à celles du réseau MLP à nombre de neurones cachés semblable.
5. Jugez des qualités de ce réseau pour un usage en simulation. Commentez

³Le terme "structuré" est en général utilisé pour signifier que l'on prend en compte les connaissances *a priori* que l'on a du système. Cela peut se caractériser par une fonction coût particulière dans l'apprentissage, imposer une partie des coefficients, une structure particulière des matrices de poids etc. On peut remarquer que le système a une structure particulière. En effet si l'on se penche sur le schéma et en notant $x_k^{(i)}$ la i -ème composante de x_k :

$$\begin{aligned} x_{k+1}^{(1)} &= f_1(x_k^{(1)}, x_k^{(3)}, u_k^{(1)}) \\ x_{k+1}^{(2)} &= f_2(x_k^{(2)}, x_k^{(4)}, u_k^{(2)}) \\ x_{k+1}^{(3)} &= f_2(x_k^{(3)}, u_k^{(2)}) \\ x_{k+1}^{(4)} &= f_2(x_k^{(4)}, u_k^{(1)}) \end{aligned}$$

5 Synthèse d'un contrôleur LQ

On veut désormais utiliser le modèle pour synthétiser un contrôleur LQ. Pour cela, il nous faut un modèle linéaire. On va linéariser le premier réseau (MLP) autour d'une position d'équilibre.

1. Formellement, linéariser le modèle $x_{k+1} = \phi(x_k, u_k)$ autour de (x_{ref}, u_{ref}) par un développement de Taylor à l'ordre 1.
2. Calculer numériquement les dérivées partielles du modèle neuronal autour de la position $x_{ref} = \begin{pmatrix} 0.6 \\ 0.8 \\ 0.4334 \\ 1.1449 \end{pmatrix}$ et $u_{ref} = 10^{-4} \begin{pmatrix} 5.972 \\ 4.505 \end{pmatrix}$.
3. Synthétiser un contrôleur LQ en utilisant la règle de Bryson. *Indice*: utiliser la fonction `dlqr` de Matlab.
4. Utiliser le contrôleur linéarisé en boucle fermée. Il vous est fourni une fonction `phi` qui vous permet de simuler le système 4 réservoirs. En pratique vous ne disposeriez pas de cette fonction, il faudrait soit tester le contrôleur sur le système réel soit utiliser le modèle que vous avez appris pour simuler le système en boucle fermée. La fonction `phi` est une fonction créée par casadi qui profite du caractère creux des matrices pour accélérer ses processus d'optimisation. Pour renvoyer une matrice pleine et convertir en un type matlab utilisez la fonction `full`.

6 Bonus : utilisation du modèle par une MPC

Un contrôleur avancé dit "MPC" pour "Model Predictive Control" a été déjà implémenté sur le système précédent et nous voulons pouvoir comparer les temps de calculs entre l'utilisation d'un modèle physique (basé sur les équations de Toricelli voir Identification) et le réseau de neurones entraîné précédemment. En effet la MPC est un contrôleur puissant et de plus en plus utilisé dans l'industrie, toutefois comme son nom l'indique, il a besoin d'un modèle pour fonctionner. D'un modèle de prédiction à N pas plus précisément qu'il faut simuler pour calculer une commande optimale à chaque pas d'échantillonnage. Un des principaux enjeux est notamment d'accélérer le temps de calcul de la MPC et donc indirectement d'accélérer la simulation du modèle.

À vous de jouer pour remplacer le modèle interne de la MPC par le modèle que vous avez appris dans les parties précédentes (modèle structuré).

1. Écrire dans le formalisme de Casadi le modèle à base de réseaux de neurones. Pour cela il suffit de stocker dans la variable `x_next` le calcul de l'état calculé par : $x_{k+1} = R_\theta(x_k, u_k)$
2. Puis de construire la MPC dont le prototype est le suivant :

```
function obj = MPC(phi, N, Q, R, P, xmin, xmax, umin, umax)
```
3. Et enfin de simuler le contrôleur en boucle fermée comme précédemment.

Bonne chance ! *Indices*:

- un réseau entraîné a la propriété `Layers` qui contient les couches du réseau.
- les poids des neurones sont stockés dans des matrices.

Exemple pour un perceptron multicouches avec pour fonction d'activation *tanh* :

```
x = MX.sym('x', 4);
u = MX.sym('u', 2);

l = net.Layers; % net est la variable correspondant au MLP entraine
full1 = l(2).Weights*[x; u] + l(2).Bias; % sortie de la couche "fully connected"
hidden = tanh(full1); % sortie de la couche cachee : fonction d'activation
x_next = l(4).Weights*hidden+l(4).Bias; % derniere couche "fully connected"
```